

Reviewer Pack — Minimal Kostant Section Dimension and SAT Clause Subset Comp...

Entry 4d2d4b3e1041 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 13:31:14 UTC

Minimal Kostant Section Dimension and SAT Clause Subset Complexity Bound

Entry ID: 4d2d4b3e1041 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 13:31:14 UTC

1. Conjecture

Field A (mathematical branch): Lie Algebra Representation Theory (Kostant Sections) **Field B** (complexity object): Boolean Satisfiability (SAT) Clause Subsets

Statement:

For every Boolean formula φ with m clauses, the minimal dimension of a Kostant section associated with its Hermitian form is bounded by $m^{1/3}$, i.e., $\dim(\text{Kostant}(\varphi)) = O(m^{1/3})$. Equivalently: The complexity measure ' m ' (number of clauses) in SAT is upper-bounded by the cube root of the dimension of the Kostant section for the corresponding Hermitian form.

Rationale (proposer's reasoning):

Kostant sections in Lie algebra representation theory are known to have a geometric interpretation, and their dimensions can be related to the properties of representations. By linking the dimension of these sections with the clause subset complexity of SAT formulas, this conjecture suggests a potential way to leverage geometric properties for understanding satisfiability problems.

Taxonomy category: LieAlgebraRepresentation (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2ed0d936ded30801

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the dimension of the Kostant section and the number of clauses (m) over 30 random seeds is greater than or equal to 0.8, with a mean difference from $m^{1/3}$ less than 3 standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Kostant sections" AND "SAT clause subsets" AND complexity - "Hermitian form" IN BOOLEAN Satisfiability AND "dimension bound" - "minimal dimension" IN Lie Algebra AND "cube root of clauses"

Top relevant hits considered: - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras - [http://arxiv.org/abs/0903.3848v1] Join-irreducible Boolean functions - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [http://arxiv.org/abs/2001.07570v2] Hom 3-Lie-Rinehart Algebras - [http://arxiv.org/abs/1212.5380v2] On properties of principal elements of Frobenius Lie algebras - [http://arxiv.org/abs/1401.3795v2] Relationship between Nichols braided Lie algebras and Nichols algebras

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(m):
        clauses = []
        for _ in range(m):
            literals = [random.choice([1, -1]) * i for i in range(1, m+1)]
            clauses.append(literals)
        return clauses

    def hermitian_form(clauses):
        n = len(clauses[0])
        H = [[0] * n for _ in range(n)]
        for clause in clauses:
            for i in range(n):
                for j in range(i, n):
                    H[i][j] += sum(c * d for c, d in zip(clause, clause[j:]))
                    H[j][i] = H[i][j]
        return H
```

```

def kostant_section_dimension(H):
    n = len(H)
    I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    A = [H[i] + I[i] for i in range(n)]
    det_A = determinant(A)
    return det_A

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += (-1) ** j * matrix[0][j] * determinant(submatrix)
    return det

def cube_root(m):
    return round(m ** (1/3))

m_values = [5, 10, 15, 20, 30, 40]
results = []
for m in m_values:
    formula = generate_formula(m)
    H = hermitian_form(formula)
    dim_Kostant = kostant_section_dimension(H)
    ratio = dim_Kostant / m
    expected_ratio = cube_root(m) / m
    results.append({
        "m": m,
        "dim_Kostant": dim_Kostant,
        "ratio": ratio,
        "expected_ratio": expected_ratio
    })

mean_ratio = sum(result["ratio"] for result in results) / len(results)
std_ratio = math.sqrt(sum((result["ratio"] - mean_ratio) ** 2 for result in results) / len(results))
support_fraction = sum(abs(result["ratio"] - result["expected_ratio"]) < 3 * std_ratio for result in results) / len(results)

return {
    "metric_name": "Ratio of Kostant Section Dimension to Number of Clauses",
    "metric_value": mean_ratio,
    "instances_tested": len(results),
    "n_max": max(result["m"] for result in results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else f"Ratio {mean_ratio} not within 3 std"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"n_max\": {trial_result['n_max']}, \"conjecture_holds\": {trial_result['conjecture_holds']}, \"counterexample\": \"{trial_result['counterexample']}\"}}")

```

```

results.append(trial_result)

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Ratio not within 3 std of expected\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete the required analysis. | next: Re-run the test with a longer timeout or increase system resources to ensure it can complete the analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18616
2	propose	ollama_remote	glm4:latest	0	0	12702
3	preregistration	ollama_remote	glm4:latest	0	0	12249
4	novelty	ollama_remote	glm4:latest	0	0	8298
5	novelty	ollama_remote	glm4:latest	0	0	10069
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35897
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7744
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6879
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12442
10	judge	ollama_remote	glm4:latest	0	0	17648

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142543 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/4d2d4b3e1041.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4d2d4b3e1041.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4d2d4b3e1041.tar.gz (if generated)